# Tutorial 7: Compare Feature Extraction vs. Fine-Tuning step by step

*Name: Laiba Niazi*

*Roll Number: 536672*

**1. Objective**

The goal of this tutorial was to implement and compare two common transfer learning approaches using pre-trained deep learning models. The performance metric used for comparison is the classification accuracy on the CIFAR-10 test dataset.

**2. Methodology**

**Data Preprocessing**

- **Dataset:** CIFAR-10 (60,000 images, 10 classes).
- **Optimization:** Images were kept at their original size (rather than resizing to 224x224) to significantly reduce training time.
- **Normalization:** Applied ImageNet-standard normalization: Mean (0.485, 0.456, 0.406) and Std Dev (0.229, 0.224, 0.225).
- **Batch Size:** 128.

**Model Configurations**

| **Feature** | **ResNet50 (Feature Extraction)** | **VGG16 (Fine-Tuning)** |
|---|---|---|
| **Pre-trained Weights** | IMAGENET1K_V1 | IMAGENET1K_V1 |
| **Strategy** | Freeze all layers, replace classifier head. | Freeze early layers, unfreeze last 5 feature layers. |
| **Learning Rate** | $1 \times 10^{-3}$ (Higher) | $1 \times 10^{-5}$ (Lower/Gentle) |
| **Optimizer** | Adam | Adam |

```python
# Load Pretrained ResNet50
model_vgg_ft = models.vgg16(weights='IMAGENET1K_V1')

# Freeze all layers initially
for param in model_vgg_ft.parameters():
    param.requires_grad = False

# Task: Unfreeze the last 5 layers
for param in list(model_vgg_ft.features.parameters())[-5:]:
    param.requires_grad = True

# Match the classifier to CIFAR-10 classes
model_vgg_ft.classifier[6] = nn.Linear(4096, 10)
model_vgg_ft = model_vgg_ft.to(device)

print("ResNet50 Feature Extraction model ready.")
```

[15] ✓ 0.9s Python

```
ResNet50 Feature Extraction model ready.
```

```python
# Load Pretrained VGG16
model_vgg_ft = models.vgg16(weights='IMAGENET1K_V1')

# Freeze all layers initially
for param in model_vgg_ft.parameters():
    param.requires_grad = False

# Task: Unfreeze the last 5 layers
for param in list(model_vgg_ft.features.parameters())[-5:]:
    param.requires_grad = True

# Match the classifier to CIFAR-10 classes
model_vgg_ft.classifier[6] = nn.Linear(4096, 10)
model_vgg_ft = model_vgg_ft.to(device)

print("VGG16 Fine-Tuning model ready.")
```

[16] ✓ 0.8s Python

```
VGG16 Fine-Tuning model ready.
```

## 3. Training Process

Both models were trained for **2 epochs**.

```
--- Starting: ResNet50 (Feature Extraction) ---
[ResNet50 (Feature Extraction)] Epoch 1, Batch 100: loss 1.848
[ResNet50 (Feature Extraction)] Epoch 1, Batch 200: loss 1.611
[ResNet50 (Feature Extraction)] Epoch 1, Batch 300: loss 1.563
[ResNet50 (Feature Extraction)] Epoch 2, Batch 100: loss 1.453
[ResNet50 (Feature Extraction)] Epoch 2, Batch 200: loss 1.461
[ResNet50 (Feature Extraction)] Epoch 2, Batch 300: loss 1.490
RESULT: Accuracy of ResNet50 (Feature Extraction) is 50.92%

--- Starting: VGG16 (Fine-Tuning) ---
[VGG16 (Fine-Tuning)] Epoch 1, Batch 100: loss 2.401
[VGG16 (Fine-Tuning)] Epoch 1, Batch 200: loss 2.052
[VGG16 (Fine-Tuning)] Epoch 1, Batch 300: loss 1.738
[VGG16 (Fine-Tuning)] Epoch 2, Batch 100: loss 1.392
[VGG16 (Fine-Tuning)] Epoch 2, Batch 200: loss 1.320
[VGG16 (Fine-Tuning)] Epoch 2, Batch 300: loss 1.243
RESULT: Accuracy of VGG16 (Fine-Tuning) is 63.42%
```

## 4. Results & Analysis

### Performance Summary

The final test accuracies recorded after 2 epochs are as follows:

- **ResNet50 (Feature Extraction):** 50.92%
- **VGG16 (Fine-Tuning):** 63.42%

### Observations

1. **Superiority of Fine-Tuning:** The VGG16 model outperformed the ResNet50 feature extractor by approximately **12.5%**. This suggests that for CIFAR-10, allowing the model to slightly adjust its internal feature detectors (the last 5 layers) is more effective than just training a new classifier on top of frozen features.
2. **Training Speed:** By omitting the Resize((224, 224)) transform, the training batches were processed much faster, making the 2-epoch comparison efficient.
3. **Loss Convergence:** The Fine-Tuning model started with a higher initial loss but showed a steeper decline, indicating that the lower learning rate was effective for the unfrozen layers.

## 5. Conclusion

While **Feature Extraction** is faster and requires less computational power (since fewer gradients are calculated), **Fine-Tuning** provided a significant boost in accuracy. For complex tasks where the target dataset differs from the original pre-training data, unfreezing a portion of the network is generally recommended.

```
Final Results Comparison:
ResNet50 FE: 50.92%
VGG16 FT: 63.42%
```